

Práctica - Analizador del Lenguaje P3 (Semántico)



Procesadores del lenguaje Curso 2025/26

Nombre, apellidos y NIA	
Alumna 1:	Ana Grima Vázquez de Prada (100495785) 100495785@alumnos.uc3m.es Grupo 80 - Grado en Ingeniería Informática
Alumna 2:	Alicia Mei Garcia Morin (100498828) 100498828@alumnos.uc3m.es Grupo 80 - Doble grado en Ingeniería Informática y ADE

Índice

Declaración de uso de IA.....	2
1. Introducción.....	3
2. Diseño del analizador léxico.....	3
2.1. Integración con el analizador léxico y sintáctico.....	3
2.2. Flujo general de ejecución.....	4
3. Gramática final.....	4
4. Tabla de símbolos y gestión de tipos.....	7
4.1. Tabla de variables.....	7
4.2. Tabla de registros.....	8
4.3. Tabla de funciones.....	9
4.4. Representación de tipos.....	9
5. Comprobaciones semánticas implementadas.....	10
5.1. Declaración y uso de variables.....	10
5.2. Comprobación de tipos en asignaciones.....	10
5.3. Comprobación de tipos en expresiones.....	11
5.4. Conversión automática de tipos.....	11
5.5. Control de flujo (if, while, break, return).....	12
5.6. Uso correcto de funciones.....	12
5.7. Acceso a campos de registros.....	13
6. Manejo de errores semánticos.....	13
7. Parte opcional implementada.....	14
8. Batería de pruebas.....	15
9. Conclusión.....	18

Declaración de uso de IA

Durante el desarrollo de esta práctica se ha hecho uso puntual de herramientas de Inteligencia Artificial como apoyo complementario al proceso de aprendizaje y desarrollo. En concreto, se ha utilizado ChatGPT para: revisar y mejorar la redacción de la memorias, resolver dudas conceptuales relacionadas con el funcionamiento del análisis semántico y la generación de código intermedio, aclarar cuestiones específicas sobre el comportamiento de la gramática, la gestión de tipos, el manejo de errores semánticos y la interpretación del enunciado de la práctica, así como apoyar el proceso de depuración mediante la identificación de posibles errores y la explicación de su causa.

Asimismo, se ha utilizado GitHub Copilot en Visual Studio Code como herramienta de apoyo para agilizar la escritura de código repetitivo y sugerir estructuras básicas de implementación.

En todo momento, estas herramientas se han empleado como soporte y recurso de aprendizaje. El diseño, comprensión, implementación y validación final del compilador (incluyendo los analizadores léxico, sintáctico y semántico, así como la generación de código intermedio) han sido realizados por nosotras. Por tanto, la autoría y el entendimiento completo del código presentado no se ven comprometidos por el uso de dichas herramientas.

1. Introducción

El objetivo de esta práctica ha sido completar el desarrollo del compilador del lenguaje Lava mediante la implementación de la fase de análisis semántico, continuando el trabajo iniciado en las prácticas anteriores. La práctica se ha desarrollado en tres entregas acumulativas:

- P1 (Analizador léxico): reconocimiento de tokens mediante expresiones regulares con `ply.lex`.
- P2 (Analizador sintáctico): definición de la gramática y construcción del AST con `ply.yacc`.
- P3 (Analizador semántico - esta entrega): comprobación de tipos, gestión de tablas de símbolos y generación de código intermedio.

En esta práctica se ha implementado un sistema de comprobación semántica integrado con el analizador sintáctico desarrollado previamente utilizando la librería `PLY`. Este sistema mantiene una serie de estructuras internas, principalmente tablas de símbolos, que almacenan información sobre variables, registros y funciones declaradas en el programa. A partir de esta información, el analizador puede verificar que cada identificador esté correctamente definido, que las expresiones utilicen tipos compatibles y que las estructuras de control se utilicen de forma adecuada.

Además, se ha desarrollado la generación de código intermedio como parte opcional para mejorar la robustez del compilador. En conjunto, el resultado final es un analizador semántico completo, capaz de detectar errores de tipo, uso indebido de identificadores y problemas de coherencia en el flujo del programa, proporcionando mensajes de error claros que facilitan la depuración de los programas escritos en el lenguaje Lava.

2. Diseño del analizador léxico

2.1. Integración con el analizador léxico y sintáctico

El analizador semántico desarrollado en esta práctica se ha diseñado como una extensión del compilador construido en las prácticas anteriores, reutilizando tanto el analizador léxico como el analizador sintáctico previamente implementados. De este modo, el flujo general del sistema sigue el modelo clásico de compilación: Código fuente → Analizador léxico → Analizador sintáctico → Analizador semántico → Resultados y estructura internas.

El sistema desarrollado mantiene el mismo diseño utilizado en las prácticas anteriores: reutilización del código existente, separación clara entre fases del compilador, integración progresiva de nuevas funcionalidades y coherencia con la especificación del lenguaje.

El punto de entrada es `main.py`, donde:

- `run_lexer_mode`, invoca el lexer directamente y vuelca los tokens en un fichero `.token` (modo `--token`).
- `run_full_mode`, que delega en `lava_parser.parse_file()`, que internamente instancia el lexer, llama a `ply.yacc` y, sobre el AST resultante, ejecuta el analizador semántico.

El parser importa el módulo `lexer` y lo pasa a `ply.yacc` como lexer externo, de modo que el analizador sintáctico nunca invoca el lexer manualmente. Todo el flujo de tokens queda encapsulado por `PLY`. Cuando el análisis completo termina sin errores, se generan cuatro ficheros de salida junto al fichero fuente: `.symbols`, `.records`, `.functions` y `.quartets`.

El módulo `parser.py` centraliza tres grandes bloques:

- Nodos del AST: `dataclasses` Python (`Program`, `FunctionDecl`, `VarDecl`, `BinaryExpr`, etc.) que se instancian en las acciones de la gramática. Al separar la representación del programa de las reglas `PLY`, el código semántico queda limpio y el AST es fácilmente inspeccionable.
- Reglas gramaticales `PLY`: funciones `p_*` que construyen el AST. No realizan comprobaciones semánticas directamente y se limitan a estructurar los datos.
- Clase `Analyzer`: recorre el AST en profundidad, gestiona las tablas de símbolos, comprueba los tipos y emite cuartetos.

2.2. Flujo general de ejecución

El funcionamiento del analizador semántico se basa en el modelo de traducción dirigida por sintaxis visto en clase, donde cada producción de la gramática tiene asociadas reglas que calculan atributos o realizan comprobaciones sobre los símbolos implicados. El flujo de ejecución del sistema sigue los siguientes pasos:

1. Se lee el programa fuente.
2. El analizador léxico genera tokens.
3. El analizador sintáctico reconoce las estructuras del lenguaje.
4. Una vez construido el AST, se ejecuta el analizador semántico, que recorre el árbol en profundidad realizando las comprobaciones necesarias.
5. Estas acciones actualizan las tablas de símbolos y realizan comprobaciones.
6. Si se detecta un error, se genera un mensaje.
7. Si no hay errores, se produce la salida final del sistema.

3. Gramática final

En este apartado se presenta la gramática final en notación BNF del lenguaje Lava utilizada en el analizador sintáctico y semántico, donde el símbolo $\rightarrow$ indica producción y el símbolo λ representa la cadena vacía. El símbolo inicial de la gramática es: *program*. Por simplicidad, se presentan directamente las reglas de producción en notación BNF.

$top_list \rightarrow top_list\ top_item \mid empty$

$top_item \rightarrow SEMICOLON$
| *basic_type* *ID* *top_basic_typed_tail*
| *ID* *ID* *top_record_typed_tail*
| *VOID* *ID* *LPAREN* *parameter_list_opt* *RPAREN* *block*
| *non_typed_simple_statement* *SEMICOLON*
| *compound_statement*
| *record_declaration* *SEMICOLON*

$top_basic_typed_tail \rightarrow LPAREN\ parameter_list_opt\ RPAREN\ block$
| $ASSIGN\ expression\ SEMICOLON$
| $id_list_tail\ SEMICOLON$

$top_record_typed_tail \rightarrow LPAREN\ parameter_list_opt\ RPAREN\ block$
| $ASSIGN\ expression\ SEMICOLON$
| $SEMICOLON$

$block \rightarrow LBRACE\ block_item_list\ RBRACE$

$block_item_list \rightarrow block_item_list\ block_item\ | empty$

$block_item \rightarrow SEMICOLON$
| $simple_statement\ SEMICOLON$
| $compound_statement$

$basic_type \rightarrow INT$
| $FLOAT$
| $CHAR$
| $BOOLEAN$

$type_spec \rightarrow basic_type\ | ID$

$simple_statement \rightarrow declaration_statement\ | non_typed_simple_statement$

$non_typed_simple_statement \rightarrow id_started_statement$
| $print_statement$
| $return_statement$
| $break_statement$

$id_started_statement \rightarrow ID\ id_statement_tail$

$id_statement_tail \rightarrow LPAREN\ argument_list_opt\ RPAREN\ | assignable_tail\ ASSIGN\ expression$

$assignable_tail \rightarrow DOT\ ID\ assignable_tail\ | empty$

$compound_statement \rightarrow block$
| $if_statement$
| $while_statement$
| $do_while_statement$

$declaration_statement \rightarrow basic_type\ ID\ declaration_suffix$
| $ID\ ID\ ASSIGN\ expression$
| $ID\ ID$

$declaration_suffix \rightarrow ASSIGN\ expression\ | id_list_tail$

$id_list_tail \rightarrow COMMA\ ID\ id_list_tail\ | empty$

print_statement → *PRINT LPAREN argument_list_opt RPAREN*

return_statement → *RETURN expression | RETURN*

break_statement → *BREAK*

if_statement → *IF LPAREN expression RPAREN block | IF LPAREN expression RPAREN block ELSE block*

while_statement → *WHILE LPAREN expression RPAREN block*

do_while_statement → *DO block WHILE LPAREN expression RPAREN SEMICOLON*

record_declaration → *RECORD ID LPAREN field_list_opt RPAREN*

field_list_opt → *field_list | empty*

field_list → *field | field_list COMMA field*

field → *type_spec ID*

parameter_list_opt → *parameter_list | empty*

parameter_list → *parameter | parameter_list COMMA parameter*

parameter → *type_spec ID*

argument_list_opt → *argument_list | empty*

argument_list → *expression | argument_list COMMA expression*

expression → *expression OR expression*
| *expression AND expression*
| *expression EQ expression*
| *expression GT expression*
| *expression GE expression*
| *expression LT expression*
| *expression LE expression*
| *expression PLUS expression*
| *expression MINUS expression*
| *expression TIMES expression*
| *expression DIVIDE expression*
| *NOT expression*
| *PLUS expression*
| *MINUS expression*
| *postfix_expression*

postfix_expression → *primary_expression*
| *postfix_expression* LPAREN *argument_list_opt* RPAREN
| *postfix_expression* DOT ID

primary_expression → LPAREN *expression* RPAREN
| INT_VALUE
| FLOAT_VALUE
| CHAR_VALUE
| TRUE
| FALSE
| ID
| NEW ID LPAREN *argument_list_opt* RPAREN

empty → λ

4. Tabla de símbolos y gestión de tipos

4.1. Tabla de variables

La tabla de variables es la estructura encargada de almacenar toda la información relacionada con las variables declaradas en el programa. Cada vez que se procesa una declaración de variable, el analizador semántico registra su información en esta tabla, para posteriormente comprobar su uso correcto. Cada entrada de la tabla contiene, al menos, los siguientes datos:

- Nombre de la variable
- Tipo de dato
- Valor asociado (cuando puede calcularse en tiempo de compilación)
- Ámbito de la variable

El objetivo principal de esta tabla es permitir realizar comprobaciones semánticas fundamentales, como por ejemplo: verificar que una variable no se declara más de una vez, comprobar que una variable se utiliza únicamente después de haber sido declarada, comprobar que el valor asignado es compatible con el tipo de la variable o permitir recuperar el tipo de una variable durante el análisis de expresiones. Por ejemplo, en un fragmento de código como:

```
int a = 5;  
float b;  
b = a;
```

El sistema registra primero la variable *a* como un entero y posteriormente la variable *b* como un número real. Cuando se procesa la asignación *b = a*, el analizador consulta la tabla de variables para comprobar los tipos implicados y verificar que la conversión es válida.

Las variables se almacenan mediante la clase *Scope*, que implementa un encadenamiento de ámbitos usando un puntero *parent*. Cada función crea su propio *Scope* enlazado al *global_scope*. La resolución de un nombre se realiza recorriendo la cadena desde el ámbito más interno hacia el

global. Esta lógica se implementa mediante reglas de comprobación que comparan el tipo de origen y el tipo de destino antes de aceptar una asignación.

Cada entrada del diccionario *symbols* guarda el tipo y, cuando es calculable estáticamente (programa sin estructuras de control ni funciones), también el valor. Este valor se usa tanto para el análisis semántico de constantes literales como para volcar el fichero *.symbols* con el formato *NOMBRE:TIPO, VALOR* requerido por el enunciado.

El flag *has_control_or_functions* del Analyzer controla el formato de salida: si el programa contiene estructuras de control o declaraciones de funciones, los valores en tiempo de compilación no son deterministas, por lo que el fichero *.symbols* solo incluye el tipo (*NOMBRE:TIPO*).

4.2. Tabla de registros

El analizador semántico mantiene una tabla específica destinada a almacenar la definición de cada registro declarado en el programa.

Cada entrada de esta tabla incluye:

- Nombre del registro
- Lista de campos
- Tipo de cada campo

Esta información permite verificar operaciones relacionadas con registros, como: comprobar que un registro está definido antes de ser utilizado, comprobar que los campos accedidos existen realmente, verificar que el número y tipo de argumentos en la instanciación son correctos y permitir el acceso a los tipos de los campos mediante el operador punto.

Según la especificación del lenguaje, los registros se instancian normalmente mediante el operador *new*, que permite inicializar todos sus campos en el momento de la creación. Por ejemplo en:

```
record Point(float x, float y);  
Point p = new Point(1.0, 2.0);
```

El analizador semántico registra el tipo *Point* en la tabla de registros, almacena los campos *x* y *y* con sus tipos, verifica que la instanciación utiliza el número correcto de argumentos y comprueba que los tipos coinciden. Además, esta tabla permite gestionar accesos encadenados a campos, como *p.x*. En estos casos, el sistema consulta la definición del registro correspondiente para validar la existencia del campo solicitado.

Los registros se almacenan en *Analyzer.records*: un diccionario que mapea el nombre del registro a una lista ordenada de pares (*nombre_campo, tipo_campo*). La estructura de lista (en lugar de diccionario) preserva el orden de declaración, necesario para validar la posición de los argumentos en *new RecordType(...)*. Un pre-scan inicial (*_pre_scan_records*) recorre el AST antes del análisis principal para procesar las declaraciones de registros. En este recorrido se comprueba que no existan registros duplicados, que no haya campos repetidos dentro de un mismo registro y que los tipos utilizados en sus campos existan previamente. De este modo, se respeta la

restricción del lenguaje Lava de que un campo de tipo registro solo puede hacer referencia a un registro ya definido anteriormente en el programa.

4.3. Tabla de funciones

El analizador semántico tiene una estructura destinada a registrar la información asociada a cada función definida en el programa. Cada entrada de esta tabla contiene:

- Nombre de la función
- Lista de parámetros
- Tipo de cada parámetro
- Tipo de retorno

Esta información permite realizar comprobaciones esenciales durante el análisis del programa, como: comprobar que una función no se declara más de una vez con la misma firma, verificar que una función existe antes de ser llamada, comprobar que el número de argumentos es correcto, comprobar que los tipos de los argumentos son compatibles y verificar que la instrucción `return` devuelve un valor del tipo correcto.

Las funciones se registran en la tabla de funciones después de validar su declaración y analizar su cuerpo. Por tanto, una función solo puede ser invocada si ya ha sido declarada anteriormente en el programa. Esta decisión respeta la especificación del lenguaje Lava, que no permite utilizar funciones antes de su declaración y, como consecuencia, tampoco permite la recursividad directa. Por ejemplo, en:

```
float square(float f) {  
    return f * f;  
}
```

El analizador analiza primero el cuerpo de la función, comprueba que el parámetro `f` existe en su ámbito local y verifica que la expresión del `return` coincide con el tipo declarado. Si todo es correcto, registra finalmente la función `square` en la tabla de funciones.

Las funciones se almacenan en *Analyzer.functions*: un diccionario que mapea el nombre a una lista de overloads. Cada overload es un diccionario con los campos *params* (lista de pares nombre-tipo), *param_types* (tupla de tipos para comparación rápida) y *return_type*.

Al registrar una nueva función se comprueba que no existe ya con exactamente la misma tupla de tipos de parámetro. Dos funciones con el mismo nombre pero distinto número o tipo de argumentos son perfectamente legales (sobrecarga). Sin embargo, dos funciones con idéntica firma pero diferente tipo de retorno se rechazan con un error semántico.

4.4. Representación de tipos

Cada expresión del programa tiene asociado un tipo, que se utiliza para verificar la compatibilidad entre operandos y operadores. Por ejemplo, en:

```
int a = 5;  
float b;
```

```
b = a + 3;
```

a es de tipo int, 3 es de tipo int, la operación $a + 3$ produce un int, la asignación a b requiere un float, se aplica una conversión automática de int a float.

El sistema de tipos de Lava se representa con strings simples: int, float, char (convertible a tipos numéricos), boolean (no convertible), void y NombreRecord.

La constante `NUMERIC_TYPES = {'int', 'float', 'char'}` agrupa los tipos sobre los que se pueden aplicar operaciones aritméticas y comparativas (tras la conversión automática correspondiente). `BASIC_TYPES = {'int', 'float', 'char', 'boolean'}` distingue los tipos primitivos de los registros definidos por el usuario.

5. Comprobaciones semánticas implementadas

5.1. Declaración y uso de variables

Una de las primeras comprobaciones semánticas implementadas consiste en verificar que las variables utilizadas en el programa estén correctamente declaradas antes de ser usadas y que no se declaren varias veces en el mismo ámbito. El analizador semántico realiza las siguientes comprobaciones:

- Declaración previa obligatoria: toda referencia a un nombre se resuelve llamando a `scope.resolve()`. Si el nombre no existe en ningún ámbito de la cadena, se lanza `SemanticError`. Esto garantiza que no se utilicen variables sin haber sido previamente declaradas.
- Prohibición de re-declaración: la declaración de una variable llama a `scope.declare()`, que comprueba si el nombre ya existe en el ámbito actual (no en los padres). Re-declarar en el mismo ámbito es un error. En cambio, una función puede declarar una variable local con el mismo nombre que una variable global.
- Valor por defecto: cuando se declara una variable sin inicializar, se le asigna el valor por defecto de su tipo: 0 para int, 0.0 para float, '\x00' para char y False para boolean. Para registros, el valor por defecto es un diccionario con los valores por defecto de cada campo, de forma recursiva, que replica la estructura del registro.
- Multideclaración sin asignación: la gramática admite `int a, b;` pero no `int a = 0, b;`. En caso de multideclaración, cada nombre se registra individualmente en el ámbito con el mismo tipo y su valor por defecto.

5.2. Comprobación de tipos en asignaciones

El analizador semántico verifica que el tipo del valor asignado a una variable sea compatible con el tipo declarado para dicha variable. Para cualquier asignación `lvalue = expr`, el método `_analyze_assign` resuelve el tipo del objetivo mediante `_resolve_target` y lo compara con el tipo inferido de la expresión usando `_can_assign(src, dst)`.

Si la asignación no es compatible, se lanza `SemanticError`. Si sí lo es, el valor se convierte mediante `_convert_value` que aplica `ord()` para `char` $\rightarrow$ `int` y `float()` para `int` $\rightarrow$ `float`.

El método `_resolve_target` maneja recursivamente tanto variables simples (`VarRef`) como accesos a campos de registro (`FieldAccess`). Para cada nivel de acceso, devuelve un par getter/setter que permite leer y escribir el valor en la estructura anidada de diccionarios que representa el registro en memoria.

5.3. Comprobación de tipos en expresiones

Se verifica que los tipos de los operandos sean compatibles con el operador utilizado y determina el tipo resultante de la expresión. Por ejemplo:

```
int a = 5 + 3;           → correcto
float b = 5 * 2.5;       → correcto
boolean c = 5 + true;    → ERROR
```

La inferencia de tipos se centraliza en `_infer_expr`, que recorre el AST de forma recursiva y devuelve el par (*tipo*, *valor*). El valor puede ser `None` cuando no es determinable en tiempo de compilación (por ejemplo, cuando depende de una variable que puede cambiar durante la ejecución). Las reglas de tipado de los operadores son:

Operador	Tipos de entrada válidos	Tipo de salida
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	int, float, char (NUMERIC_TYPES)	Resultado numérico según jerarquía: float > int > char
<code>></code> <code>>=</code> <code><</code> <code><=</code>	int, float, char (NUMERIC_TYPES)	boolean
<code>==</code>	Cualquier combinación compatible	boolean
<code>&&</code> <code> </code>	boolean (ambos operandos)	boolean
<code>!</code>	boolean	boolean
<code>-</code> <code>+</code> (unario)	int, float, char	mismo tipo que el operando

5.4. Conversión automática de tipos

La jerarquía de conversiones automáticas implementada sigue lo indicado en el enunciado. El método `_numeric_result_type(left, right)` determina el tipo resultado de una operación aritmética o comparativa:

Origen	Destino	Mecanismo
char	int	<code>ord(valor)</code> : valor numérico ASCII del carácter (0-255)
char	float	<code>float(ord(valor))</code> : conversión en dos pasos
int	float	<code>float(valor)</code> : conversión directa de entero a real

boolean	cualquiera	No permitido: se lanza SemanticError
---------	------------	--------------------------------------

La jerarquía es `float > int > char`. Si en una operación binaria los dos operandos son de distinto tipo numérico, el tipo resultado es el más general de los dos, y cada operando se convierte a ese tipo antes de aplicar el operador.

5.5. Control de flujo (if, while, break, return)

Las estructuras de control se verifican semánticamente en los métodos específicos del Analyzer:

- *if / if-else*: la condición se comprueba mediante `_check_boolean_condition`, que requiere que su tipo inferido sea `boolean`. Cualquier otro tipo produce un error semántico. El *else* es opcional en la gramática y siempre se asocia al *if* más cercano, tal y como se definió en la fase sintáctica.
- *while*: igual que *if*, la condición debe ser `boolean`. El análisis del cuerpo se realiza pasando `in_loop=True` para habilitar *break*.
- *do-while*: se analiza el cuerpo antes que la condición (al contrario que *while*), pero la comprobación de tipo de la condición es idéntica.
- *break*: si `in_loop` es `False` cuando se encuentra un *break*, se lanza `SemanticError("[ERROR] break fuera de un bucle")`. En tiempo de generación de cuartetos, *break* emite un *JUMP* a la etiqueta de fin del bucle, que se gestiona mediante `break_stack`.
- *return*: en funciones no `void`, la presencia de al menos un *return* con expresión es obligatoria y el tipo de la expresión debe ser asignable al tipo de retorno declarado. En funciones `void`, se permite el uso de *return*; como salida anticipada sin valor, pero cualquier *return* con expresión es un error. Fuera de una función, *return* es siempre un error.

5.6. Uso correcto de funciones

La resolución de llamadas a funciones implementa el algoritmo de sobrecarga descrito en el enunciado con prioridad en dos fases:

1. Coincidencia exacta: se buscan sobrecargas donde el tipo de cada argumento es idéntico al tipo del parámetro correspondiente.
2. Coincidencia con conversión: si no se produjeron resultados, se buscan sobrecargas donde cada argumento es asignable (vía `_can_assign`) al parámetro correspondiente.

Si ambas fases producen más de un candidato, la llamada es ambigua y se lanza un error. Si ninguna fase produce candidatos, la función no existe o ninguna firma es compatible. Otras verificaciones sobre funciones:

- Uso antes de declaración: el analizador recorre el programa en orden. Si se llama a una función antes de su declaración, la lista de *overloads* estará vacía en ese momento y se producirá un error.

- Limitación de recursividad: como consecuencia del punto anterior, una función no puede llamarse a sí misma dentro de su propio cuerpo, ya que en el momento de analizar el cuerpo la función aún no ha sido registrada en la tabla.
- Sobrecarga válida vs. inválida: se permite que existan múltiples funciones con el mismo nombre si sus firmas (número o tipo de parámetros) difieren. No se permite que existan dos funciones con idéntica firma y distinto tipo de retorno.

5.7. Acceso a campos de registros

El acceso a campos de registros (operador punto) se verifica en `_infer_expr` para `FieldAccess` y en `_resolve_target` para el lado izquierdo de las asignaciones. En ambos casos:

- Se infiere el tipo del objeto base.
- Se comprueba que dicho tipo no es un tipo básico (los tipos básicos no tienen campos).
- Se busca el campo en el diccionario de campos del registro correspondiente. Si el campo no existe, se lanza `SemanticError`.
- El tipo del resultado es el tipo del campo encontrado.

Para el lado izquierdo, `_resolve_target` devuelve además una función `setter` que actualiza el valor directamente en la estructura de diccionarios anidados que representa el registro en memoria.

6. Manejo de errores semánticos

El sistema implementado permite detectar una amplia variedad de errores semánticos relacionados con el uso incorrecto de los elementos del lenguaje, que se clasifican en las siguientes categorías:

Categoría	Ejemplo de mensaje
Tipos incompatibles	[ERROR] No se puede asignar 'boolean' a 'float'
Variable no declarada	[ERROR] Variable 'name' no declarada
Variable ya declarada	[ERROR] Variable 'name' ya declarada
Record no declarado	[ERROR] Record 'name' no declarado
Record ya declarado	[ERROR] Record 'name' ya declarado
Campo inexistente	[ERROR] El record 'name' no tiene el campo 'x'
Función no existe	[ERROR] La función 'suma' no existe o aún no ha sido declarada
Función ya declarada	[ERROR] Función 'suma' ya declarada con la misma firma
Llamada ambigua	[ERROR] Llamada ambigua a la función 'name'
return fuera de función	[ERROR] return fuera de una función
return en función void	[ERROR] Una función void no puede devolver valor

break fuera de bucle	[ERROR] break fuera de un bucle
Condición no boolean	[ERROR] La condición del label debe ser boolean
Operador sobre tipo inv.	[ERROR] El operador 'op' requiere operandos numéricos/char

El manejo de errores semánticos se basa en la excepción `SemanticError`. Cuando el analizador detecta una inconsistencia, lanza esta excepción con un mensaje descriptivo. En *main.py*, dicha excepción se captura junto con los errores sintácticos, se imprime el mensaje correspondiente y se detiene la ejecución. Esto permite informar de forma clara del primer error detectado. El formato general de los mensajes sigue el siguiente patrón: ERROR: descripción del error.

7. Parte opcional implementada

Se ha implementado la parte opcional de generación de código intermedio en cuartetos (código de tres direcciones), pese a no haberlo visto en clase todavía. Para ello, hemos usado distintos recursos: vídeos de YouTube, apuntes de la asignatura y resolución de dudas con la IA.

El código se genera directamente durante el recorrido principal del AST realizado por Analyzer, después del pre-scan de registros. Se genera el juego de instrucciones especificado en el enunciado, incluyendo:

Instrucciones	Arg 1	Arg 2	Resultado	Descripción
ADD, SUB, MUL, DIV	Valor	Valor	Temporal	Operación aritmética binaria
GT, GTE, LT, LTE, EQ	Valor	Valor	Temporal	Comparación binaria
AND, OR	Valor	Valor	Temporal	Operación lógica binaria
UMINUS, UPLUS, NOT	Valor	—	Temporal	Operación unaria
CHAR_TO_INT, INT_TO_FLOAT	Valor	—	Temporal	Casting automático
ASSIGN	Valor	—	Variable	Asignación a variable
JUMPT, JUMPF	Valor	Etiqueta	—	Salto condicional (verdadero/falso)
JUMP	Etiqueta	—	—	Salto incondicional
LABEL	Etiqueta	—	—	Definición de etiqueta
PRINT	Valor	—	—	Llamada al sistema de impresión

Temporales y etiquetas utilizan el prefijo `@` para evitar colisiones con los nombres de variable del programador. Los contadores se incrementan globalmente en *temp_counter* y *label_counter*.

El enunciado especifica que no se generan cuartetos para operaciones con registros ni para funciones. El método `_emit_expr_if_supported` implementa esto retornando *False* cuando la expresión involucra *FieldAccess*, *NewExpr* o *CallExpr*. Únicamente las operaciones entre literales y variables básicas producen cuartetos.

El método `_emit_cast_ref` se encarga de insertar las instrucciones de conversión cuando el tipo de un operando no coincide con el tipo requerido. Las conversiones soportadas en código intermedio son:

- `char` → `int`: emite `CHAR_TO_INT,src,_,@Tn`
- `int` → `float`: emite `INT_TO_FLOAT,src,_,@Tn`
- `char` → `float`: emite `CHAR_TO_INT` seguido de `INT_TO_FLOAT` (conversión en dos pasos a través de un temporal intermedio)

Por ejemplo, para el siguiente fragmento de código:

```
int counter;  
float result;  
while (counter < 5) {  
    counter = counter + 1;  
    result = counter;  
    result = -result;  
}
```

El analizador genera el siguiente fichero `.quartets`:

```
ASSIGN,0,_,counter  
ASSIGN,0.0,_,result  
LABEL,@L1,_,_  
LT,counter,5,@T1  
JUMPF,@T1,@L2,_  
ADD,counter,1,@T2  
ASSIGN,@T2,_,counter  
INT_TO_FLOAT,counter,_,@T3  
ASSIGN,@T3,_,result  
UMINUS,result,_,@T4  
ASSIGN,@T4,_,result  
JUMP,@L1,_,_  
LABEL,@L2,_,_
```

8. Batería de pruebas

Al igual que en la práctica anterior, se ha desarrollado un script en Python para ejecutar automáticamente todos los programas de prueba incluidos en el conjunto de tests. Este script permite lanzar de forma secuencial el compilador sobre cada fichero de entrada y registrar los resultados obtenidos en un fichero de log.

El script recorre automáticamente el directorio de tests, ejecuta el analizador sobre cada fichero y guarda el contenido del test, la salida producida por consola, el código de salida y, cuando se generan, el contenido de los ficheros `.symbols`, `.records`, `.functions` y `.quartets`. Finalmente, todos los resultados se almacenan en un fichero de log que permite revisar fácilmente el comportamiento del sistema y verificar que cada caso de prueba produce la salida esperada.

La nomenclatura de ficheros sigue el patrón test_NN_ok.java / test_NN_error.java. La batería de pruebas se organiza en carpetas: tests_p1/ con los casos para el analizador léxico, tests_p2/ con los casos para el analizador sintáctico y tests_p3/ con los casos específicos del analizador semántico y la generación de código intermedio.

Aspecto probado	Descripción del caso	Resultado esperado
Fichero vacío o solo delimitadores	;; ;	Análisis correcto; ficheros de salida vacíos
Tabla de símbolos sin control	int a = 28; float b = 24.9; char c = 'x'; boolean d = true;	.symbols con NOMBRE:TIPO,VALOR
Literales numéricos	enteros decimales, binarios, octales, hexadecimales y floats científicos	Valores normalizados en .symbols y asignaciones en .quartets
Conversión char → int	int i = 1 + 'A';	Sin error; CHAR_TO_INT en .quartets
Conversión char → float	float f = 7.5 + c * 8;	Sin error; CHAR_TO_INT e INT_TO_FLOAT en .quartets
Conversión int → float	float b; b = a;	Sin error; INT_TO_FLOAT en .quartets cuando se genera código
Operadores booleanos y comparativos	true && false, !false, 5 < 8, 'a' == 'a'	Resultado boolean y cuartetos AND, NOT, LT, EQ
Tabla de símbolos con control	programa con if, while, do-while o función	.symbols con solo NOMBRE:TIPO
Tabla de registros simples	record Point(int x, int y); Point p; p.x = 10;	.records con Point y .symbols con estructura del registro
Tabla de registros anidados	record Line(Point a, Point b); Line l; l.a.x = 1;	.records con tipos de campo correctos y .symbols con diccionarios anidados
Variable no declarada	a = 4;	SemanticError: variable 'a' no declarada
Re-declaración de variable	int a; float a;	SemanticError: variable 'a' ya declarada
Asignación incompatible	int a; a = true;	SemanticError: no se puede asignar boolean a int
Condición no boolean en if	if (a) { ... } con a de tipo int	SemanticError: condición del if debe ser boolean
break fuera de bucle	break; en el ámbito global	SemanticError: break fuera de un bucle
return fuera de una función	return 3; en ámbito global	SemanticError: return fuera de una

		función
return incorrecto	función int que devuelve boolean o función void con return expr	SemanticError por tipo de retorno incorrecto
Función antes de declaración	square(3); antes de declarar square	SemanticError: función no existe o aún no ha sido declarada
Llamada correcta a función	int doble(int x) { return x+x; } int a = doble(3);	.functions con firma de doble y análisis correcto
Sobrecarga válida	int square(int); float square(float);	Dos entradas en .functions
Firma duplicada	int foo(int x); int foo(int x);	SemanticError: función ya declarada con la misma firma
Llamada ambigua	foo('A') con foo(int) y foo(float)	SemanticError: llamada ambigua
Argumento incompatible	foo(true) cuando solo existe foo(int)	SemanticError: no existe sobrecarga compatible
Tipo no declarado	UnknownType x;	SemanticError: tipo no declarado
Campo inexistente	p.z = 3;	SemanticError: el record no tiene ese campo
Campo duplicado en record	record Point(int x, int x);	SemanticError: campo duplicado
new con argumentos incorrectos	Point p = new Point(1);	SemanticError: número incorrecto de argumentos
Acceso a campo sobre tipo básico	int a; a.x = 10;	SemanticError: tipo básico sin campos
Recursividad directa	int f(int x) { return f(x); }	SemanticError: función no existe o aún no ha sido declarada
Recursividad no permitida en factorial	factorial llama a factorial dentro de su cuerpo	SemanticError: función no existe o aún no ha sido declarada
Cuartetos aritméticos	f1 = 5*4 - 80/10;	.quartets con MUL, DIV, SUB, ASSIGN
Cuartetos if-else	if (a < 5) { ... } else { ... }	LABEL, JUMPF, JUMP en .quartets
Cuartetos while con break	while (i < 10) { if (i == 4) break; ... }	break → JUMP a etiqueta de fin del while
Cuartetos do-while	do { i=i+1; print(i); } while (i<3);	LABEL inicio, cuerpo, condición y JUMPT al inicio
Cuartetos con castings	result = counter; con result float y counter int	INT_TO_FLOAT en .quartets

9. Conclusión

La implementación del analizador semántico ha permitido completar el desarrollo del compilador del lenguaje Lava iniciado en las prácticas anteriores. Este proceso ha puesto de manifiesto la importancia de tomar decisiones de diseño desde las primeras fases, especialmente en la definición de la gramática, pensando en su evolución futura. Asimismo, el trabajo realizado en cada entrega ha permitido identificar y corregir errores progresivamente gracias al feedback recibido.

El sistema resultante es capaz de procesar programas completos, verificando su corrección desde el punto de vista léxico, sintáctico y semántico. De esta forma, el compilador no solo reconoce la estructura del programa, sino que también comprueba que su uso de variables, tipos, funciones y estructuras de control sea coherente y válido.

Uno de los retos más relevantes de esta práctica ha sido la resolución de determinados conflictos en la gramática, en particular aquellos relacionados con construcciones que comparten un mismo prefijo. La solución adoptada consistió en reorganizar las reglas de producción para diferenciar los casos en una fase posterior del análisis, evitando así ambigüedades y manteniendo la gramática clara y mantenible.

Otro aspecto que ha resultado especialmente útil ha sido la separación entre la construcción del árbol sintáctico (AST) y el análisis semántico. Esta división de responsabilidades ha facilitado la organización del código y ha permitido implementar cada fase de forma más ordenada. Además, el uso de estructuras de datos adecuadas para representar los nodos del AST ha simplificado el recorrido del árbol y la implementación de las comprobaciones semánticas.

También habría que mencionar la decisión de realizar un recorrido previo del programa para registrar los tipos de registros antes de iniciar el análisis principal. Este enfoque ha permitido que los registros puedan utilizarse como tipos de campo independientemente del orden en el que se definan, aportando mayor flexibilidad al lenguaje sin aumentar la complejidad del sistema.

Como parte opcional, se ha implementado la generación de código intermedio en forma de cuartetos. Esta funcionalidad se ha integrado en el propio proceso de análisis semántico, aprovechando la información de tipos ya calculada. La gestión de conversiones automáticas entre tipos y el tratamiento de instrucciones de control como break han sido los aspectos más elaborados de esta fase. Para ello, hemos usado distintos recursos: vídeos de YouTube, apuntes de la asignatura y resolución de dudas con la IA.

En conjunto, la práctica ha supuesto un trabajo significativo tanto desde el punto de vista conceptual como de implementación. El resultado final es un compilador funcional que cumple todos los requisitos obligatorios del enunciado y que, además, incorpora la generación de código intermedio como mejora adicional.



Práctica elaborada por:
Alicia Mei García Morín
Ana Grima Vázquez de Prada

A fecha de:
28/04/2026

